

Data Exploration

The following tutorial contains examples of Python code for data exploration. You should refer to the "Data Exploration" chapter of the "Introduction to Data Mining" book and Lecture notes.

Data exploration refers to the preliminary investigation of data in order to better understand its specific characteristics. There are two key motivations for data exploration:

1. To help users select the appropriate preprocessing and data analysis technique used.
2. To make use of humans' abilities to recognize patterns in the data.

Read the step-by-step instructions below carefully. To execute the code, click on the cell and press the SHIFT-ENTER keys simultaneously.

Summary of toolbox (for review purposes):

1. data summary statistics: `pandas.DataFrame.describe`
(<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.describe.html>)
2. data standardization from `sklearn.preprocessing` import `StandardScaler` (<https://scikit-learn.org/stable/modules/preprocessing.html>)
3. visualization `pandas`, `matplotlib`
4. imputation of missing values (<https://scikit-learn.org/stable/modules/impute.html#impute>)

2.1. Summary Statistics

Summary statistics are quantities, such as the mean and standard deviation, that capture various characteristics of a potentially large set of values with a single number or a small set of numbers. In this tutorial, we will use the Iris sample data, which contains information on 150 Iris flowers, 50 each from one of three Iris species: Setosa, Versicolour, and Virginica. Each flower is characterized by five attributes:

- sepal length in centimeters
- sepal width in centimeters
- petal length in centimeters
- petal width in centimeters
- class (Setosa, Versicolour, Virginica)

In this tutorial, you will learn how to:

- Load a txt data file into a Pandas DataFrame object.
- Compute various summary statistics from the DataFrame.

1. First, you need to download the iris.txt from BB.

Code: The following code uses Pandas to read the TXT file and store them in a DataFrame object named data. Next, it will display the first five rows of the data frame.

4. Deal with real data

4.1 fill missing data with mean

```
import pandas as pd
ResearchData=pd.read_csv('Airbnb.csv') #load data as pandas dataframe
ResearchData.head(10) #it is seen that the forth row has NaN (missing record).
```

	id	log_price	property_type	accommodates	bathrooms	bed_type	cancellation_policy	cleaning_fee	city	host_has
0	6901257	5.010635	Apartment	3	1.0	Real Bed	strict	True	NYC	
1	6304928	5.129899	Apartment	7	1.0	Real Bed	strict	True	NYC	
2	7919400	4.976734	Apartment	5	1.0	Real Bed	moderate	True	NYC	
3	13418779	6.620073	House	4	1.0	Real Bed	flexible	True	SF	
4	3808709	4.744932	Apartment	2	1.0	Real Bed	moderate	True	DC	
5	12422935	4.442651	Apartment	2	1.0	Real Bed	strict	True	SF	
6	11825529	4.418841	Apartment	3	1.0	Real Bed	moderate	True	LA	
7	180792	4.787492	House	2	1.0	Real Bed	moderate	True	SF	
8	5385260	3.583519	House	2	1.0	Real Bed	moderate	True	LA	
9	5578513	4.605170	Apartment	2	1.0	Real Bed	strict	True	NYC	

后续步骤: [New interactive sheet](#)

ResearchData.describe()

	id	log_price	accommodates	bathrooms	number_of_reviews	review_scores_rating	bedrooms	b
count	6.813300e+04	68133.000000	68133.000000	68133.000000	68133.000000	52858.000000	68133.000000	68133.000
mean	1.125938e+07	4.774962	3.144996	1.230160	20.830258	94.012789	1.271733	1.703
std	6.083296e+06	0.711940	2.134786	0.562762	37.493498	7.832125	0.855334	1.233
min	3.440000e+02	1.609438	1.000000	0.000000	0.000000	20.000000	0.000000	0.000
25%	6.245923e+06	4.290459	2.000000	1.000000	1.000000	92.000000	1.000000	1.000
50%	1.224682e+07	4.700480	2.000000	1.000000	6.000000	96.000000	1.000000	1.000
75%	1.640494e+07	5.192957	4.000000	1.000000	23.000000	100.000000	1.000000	2.000
max	2.123090e+07	7.600402	16.000000	8.000000	532.000000	100.000000	10.000000	18.000

- ✓ 1. (20 points) What is the 50 percentile, 75 percentile of log_price?

According to the data chart above,

50 percentile of log price:4.700480

75 percentile of log price:5.192957

```
from sklearn.impute import SimpleImputer #this tool fill missing value with mean
import numpy as np
imp = SimpleImputer(missing_values=np.nan, strategy='mean')
missing_filled_new=imp.fit_transform(ResearchData[['review_scores_rating','bedrooms']])
ResearchData[['review_score_rating_filled','bedrooms_filled']]=missing_filled_new
```

ResearchData.head()

	id	log_price	property_type	accommodates	bathrooms	bed_type	cancellation_policy	cleaning_fee	city	host_has
0	6901257	5.010635	Apartment	3	1.0	Real Bed	strict	True	NYC	
1	6304928	5.129899	Apartment	7	1.0	Real Bed	strict	True	NYC	
2	7919400	4.976734	Apartment	5	1.0	Real Bed	moderate	True	NYC	
3	13418779	6.620073	House	4	1.0	Real Bed	flexible	True	SF	
4	3808709	4.744932	Apartment	2	1.0	Real Bed	moderate	True	DC	

后续步骤: [New interactive sheet](#)

2. (60 points) Explore and visualize the relationship between features=

['property_type','accommodates','bathrooms','review_scores_rating','bedrooms','bed'] and prediction target "log price".

```
feature=['property_type','accommodates','bathrooms','review_score_rating_filled','bedrooms_filled','beds']
target='log_price'
x=ResearchData[feature]
y=ResearchData[target]
x['property_type'].drop_duplicates()
```

	property_type
0	Apartment
3	House
15	Loft
16	Townhouse

dtype: object

```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(drop='first')
enc_f=enc.fit(x[['property_type']])
print(enc_f.categories_) #the function will generate four dummies, alphabetically sorted
#so the four columns indicate: ['Apartment', 'House', 'Loft', 'Townhouse']
#the first category "Apartment" will be dropped.

enc_df = pd.DataFrame(enc_f.transform(x[['property_type']]).toarray())
enc_df.columns = ['House', 'Loft', 'Town House']
enc_df.head()
```

[array(['Apartment', 'House', 'Loft', 'Townhouse'], dtype=object)]

	House	Loft	Town House
0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0
3	1.0	0.0	0.0
4	0.0	0.0	0.0

后续步骤: [New interactive sheet](#)

```
# merge with main df on key values, this will be used as IV
x = x.join(enc_df)
x = x.drop('property_type', axis=1)
x.head()
```

	accommodates	bathrooms	review_score_rating_filled	bedrooms_filled	beds	House	Loft	Town House
0	3	1.0	100.000000	1.0	1	0.0	0.0	0.0
1	7	1.0	93.000000	3.0	3	0.0	0.0	0.0
2	5	1.0	92.000000	1.0	3	0.0	0.0	0.0
3	4	1.0	94.012789	2.0	2	1.0	0.0	0.0
4	2	1.0	40.000000	0.0	1	0.0	0.0	0.0

后续步骤: [New interactive sheet](#)

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer # Import SimpleImputer

x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.30, random_state=0)

lin_reg = LinearRegression() #
lin_reg.fit(x_train, y_train) #
```

```
print(lin_reg.coef_)
print(lin_reg.intercept_)

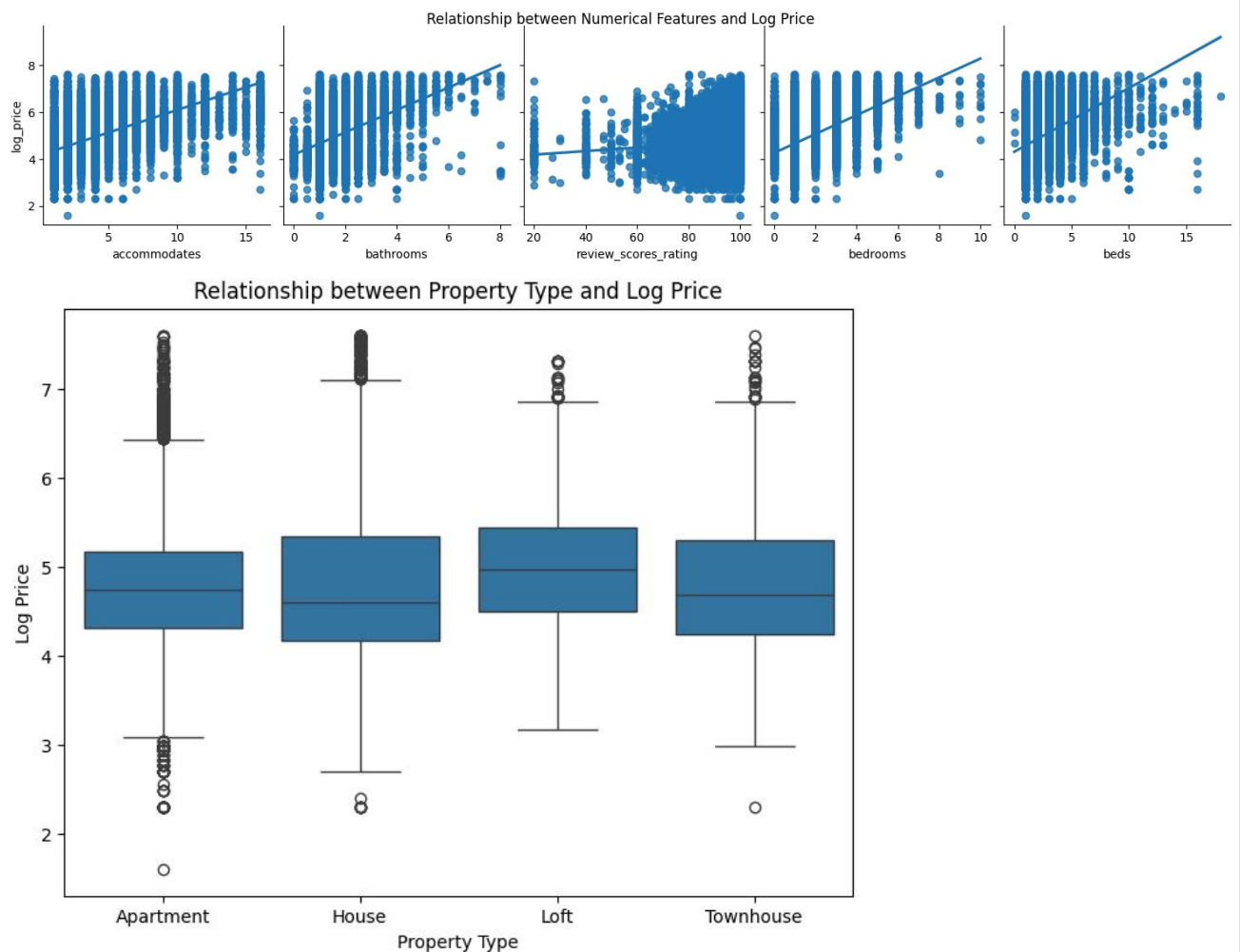
[ 0.1704748  0.13703868  0.0080227  0.12200576 -0.05398534 -0.19712244
  0.11035847 -0.18170599]
3.3044164091475214
```

- According to the coef, the coefficient of accomodate , bathrooms, review score rating, bedrooms and loft are positive, so they are positively correlated. The coefficients of the remaining variables are negative, so they are negatively correlated.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Visualize the relationship between numerical features and log_price
numerical_features = ['accommodates', 'bathrooms', 'review_scores_rating', 'bedrooms', 'beds']
sns.pairplot(ResearchData, x_vars=numerical_features, y_vars='log_price', height=3, kind='reg')
plt.suptitle('Relationship between Numerical Features and Log Price', y=1.02)
plt.show()

# Visualize the relationship between property_type and log_price
plt.figure(figsize=(8, 6))
sns.boxplot(x='property_type', y='log_price', data=ResearchData)
plt.title('Relationship between Property Type and Log Price')
plt.xlabel('Property Type')
plt.ylabel('Log Price')
plt.show()
```



5. (20 points) Import the dataset you may use for course project. Explore the relationship among at most three variables in the datasets. Different group members should use different variables for this question.

```
ResearchData1=pd.read_csv('ScreenTime vs MentalWellness.csv') #load data as pandas dataframe
ResearchData1.head(10) #
```

	user_id	age	gender	occupation	work_mode	screen_time_hours	work_screen_hours	leisure_screen_hours	sleep_hours
0	U0001	33	Female	Employed	Remote	10.79	5.44	5.35	6.63
1	U0002	28	Female	Employed	In-person	7.40	0.37	7.03	8.05
2	U0003	35	Female	Employed	Hybrid	9.78	1.09	8.69	6.48
3	U0004	42	Male	Employed	Hybrid	11.13	0.56	10.57	6.89
4	U0005	28	Male	Student	Remote	13.22	4.09	9.13	5.79
5	U0006	28	Non-binary/Other	Self-employed	Hybrid	9.83	0.53	9.30	7.19
6	U0007	42	Non-binary/Other	Retired	Remote	6.02	2.82	3.20	7.44
7	U0008	36	Male	Employed	In-person	6.21	0.75	5.46	6.78
8	U0009	26	Male	Student	In-person	6.28	0.67	5.61	8.48
9	U0010	34	Male	Employed	Hybrid	9.37	0.84	8.53	7.78

后续步骤: [New interactive sheet](#)

```
ResearchData1.describe()
```

	age	screen_time_hours	work_screen_hours	leisure_screen_hours	sleep_hours	sleep_quality_1_5	stress_level_0_10
count	400.00000	400.000000	400.000000	400.000000	400.000000	400.000000	400.000000
mean	29.77750	9.024900	2.183075	6.841825	7.013175	1.397500	8.150000
std	7.46608	2.491058	1.931321	2.220896	0.852421	0.652348	2.090000
min	16.00000	1.000000	0.110000	0.890000	4.640000	1.000000	0.000000
25%	24.00000	7.372500	0.695000	5.460000	6.397500	1.000000	6.900000
50%	30.00000	9.090000	1.455000	6.700000	7.030000	1.000000	8.800000
75%	35.00000	10.495000	3.012500	8.417500	7.640000	2.000000	10.000000
max	60.00000	19.170000	12.040000	13.350000	9.740000	4.000000	10.000000

```
from sklearn.impute import SimpleImputer #this tool fill missing value with mean
import numpy as np
imp = SimpleImputer(missing_values=np.nan, strategy='mean')
missing_filled_new = imp.fit_transform(ResearchData1[['stress_level_0_10']])
ResearchData1['stress_level_0_10_filled'] = missing_filled_new
```

```
ResearchData1.head()
```

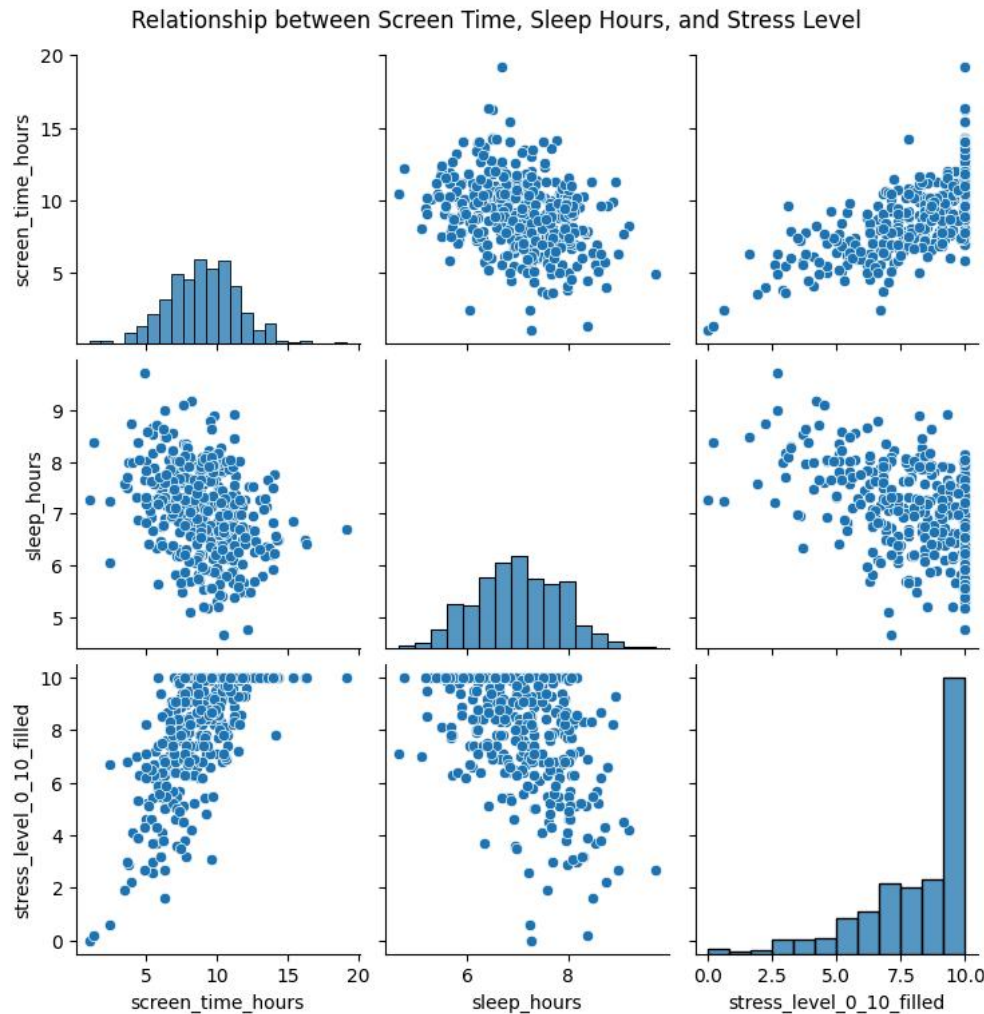
后续步骤: [New interactive sheet](#)

```
import matplotlib.pyplot as plt
import seaborn as sns

# Select the variables to visualize
variables_to_visualize = ['screen_time_hours', 'sleep_hours', 'stress_level_0_10_filled']

# Create a pair plot to visualize the relationships between the selected variables
sns.pairplot(ResearchData1[variables_to_visualize])
plt.suptitle('Relationship between Screen Time, Sleep Hours, and Stress Level', y=1.02)
```

```
plt.figure(figsize=(10, 10))
plt.subplot(3, 3, 1)
plt.title('Relationship between Screen Time, Sleep Hours, and Stress Level')
plt.show()
```



Week 3 Assignment

Provide your student ID and current datetime for this assignment

Please submit your "HTML" file

- (20 points) What is the 50 percentile, 75 percentile of log_price?
- (60 points) Explore and visualize the relationship between features=['property_type','accommodates','bathrooms','review_scores_rating','bedrooms','bed'] and prediction target "log price".
- (20 points) Import the dataset you may use for course project. Explore the relationship among at most three variables in the datasets. Different group members should use different variables for this question.